

# Chapter 1: Spring Boot Fundamentals

## 1. What is Spring?

Spring is a **Java Framework** used to build enterprise applications.

It provides solutions for:

- Web Applications
- REST APIs
- Database Access
- Security
- Dependency Injection
- Microservices
- Cloud Applications

Without Spring, developers have to configure everything manually.

### Real-world Analogy

Imagine you want to build a house.

#### Without Spring

- Buy bricks
- Lay bricks
- Mix cement
- Install wiring
- Install plumbing
- Paint walls

Everything is manual.

#### With Spring

Hire a construction company

↓

They handle everything

↓

You focus on the house design

Spring is that construction company.

## 2. Problems Before Spring Boot

Before Spring Boot, developers had to configure everything manually.

Example:

To build a simple web application you needed to configure:

- Web Server
- Servlet Container
- XML files
- Dependency Injection
- JSON
- Database
- Security
- Logging

Even a "Hello World" application required many configuration files.

Example:

- web.xml
- dispatcher-servlet.xml
- applicationContext.xml
- pom.xml
- beans.xml

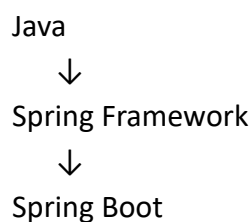
A simple project could require hundreds of lines of configuration.

## 3. What is Spring Boot?

Spring Boot is built **on top of Spring Framework**.

It removes most of the manual configuration.

Think of it like this:



Spring Boot uses Spring internally but makes development much easier.

## Definition

Spring Boot is a framework that simplifies Spring application development using **auto-configuration**, **embedded servers**, and **starter dependencies**.

## 4. Spring vs Spring Boot

| Spring Framework             | Spring Boot           |
|------------------------------|-----------------------|
| Lots of configuration        | Minimal configuration |
| Requires external server     | Embedded server       |
| XML configuration            | Annotation-based      |
| Manual dependency management | Starter dependencies  |
| Slower development           | Faster development    |
| More setup                   | Almost zero setup     |

## Example

### Without Spring Boot

```
Configure Tomcat
↓
Configure DispatcherServlet
↓
Configure Jackson
↓
Configure Database
↓
Configure Beans
↓
Run Application
```

### With Spring Boot

```
@SpringBootApplication
public class ProcessmindApiApplication {
    public static void main(String[] args) {
        SpringApplication.run(ProcessmindApiApplication.class,args);
    }
}
```

That's enough to start an enterprise application.

## 5. What Happens When Spring Boot Starts?

When you run

```
SpringApplication.run(ProcessmindApiApplication.class,args);
```

Spring Boot performs many tasks automatically.

Start JVM → Read Dependencies → Create Spring Container → Scan Packages → Create Beans → Read application.yml → Connect Database → Run Flyway → Configure Security → Start Embedded Tomcat → Application Ready

## 6. What is Auto Configuration?

One of Spring Boot's biggest features.

Spring Boot looks at your dependencies and configures them automatically.

Example:

Suppose you add

spring-boot-starter-web

Spring Boot automatically configures

- Tomcat
- JSON
- REST Controllers
- DispatcherServlet

No extra code needed.

Example :

You add

spring-boot-starter-data-jpa

Spring Boot automatically configures

- Hibernate
- EntityManager
- Transaction Manager

Example :

You add

spring-boot-starter-security

Spring Boot automatically enables security.

This is why you saw a login page.

## 7. Embedded Server

Traditional Java Web Applications required

Write Application → Build WAR file → Install Tomcat → Deploy WAR → Run Server

### Spring Boot

Run Application → Embedded Tomcat Starts → Application Ready

Tomcat is already inside your application.

You don't install it separately.

## 8. What is Maven?

Maven is a **Build Tool**.

Its jobs are

- Download libraries
- Compile Java
- Run Tests
- Build Project
- Package Project

## 9. What are Dependencies?

Dependencies are external libraries your project uses.

Example

| Technology      | Responsibility                      | Example Use                           |
|-----------------|-------------------------------------|---------------------------------------|
| Spring Web      | Handle HTTP requests and responses  | Expose REST endpoints for client apps |
| Spring Data JPA | Object-relational mapping (ORM)     | Interact with PostgreSQL/MySQL        |
| Spring Security | Authentication & authorization      | Secure APIs with JWT/OAuth2           |
| Flyway          | Version control for database schema | Manage SQL migrations automatically   |

Without dependencies,  
you would need to write these frameworks yourself.

## 10. Dependencies Used in ProcessMind AI

| Dependencies      | Purpose                                    | Example                                                       |
|-------------------|--------------------------------------------|---------------------------------------------------------------|
| Spring Web        | Creates REST APIs                          | @RestController public class UserController { }               |
| Spring Validation | Validates user input                       | @NotBlank private String username;                            |
| Spring Data JPA   | Database communication                     | userRepository.findAll(); instead of SELECT * FROM users;     |
| PostgreSQL Driver | Allows Java to communicate with PostgreSQL | Spring → Driver → PostgreSQL                                  |
| Flyway            | Database version control                   | V1 → Create Schema V2 → Create Users V3 → Create Workflow     |
| Spring Security   | Protects APIs                              | GET /users → Login Required                                   |
| Lombok            | Generate boilerplate code                  | Instead of getters, setters, constructors → write @Data       |
| Actuator          | Application monitoring                     | /actuator/health returns { "status": "UP" }                   |
| Swagger           | API Documentation                          | Automatically generates GET /users, POST /login documentation |

## 11. Request Flow

Suppose browser requests

Browser → HTTP Request → Spring Boot → Controller → Service → Repository → Hibernate → PostgreSQL → Response → Browser

## 12. Advantages

- Very little configuration
- Embedded Tomcat
- Auto Configuration
- Starter Dependencies
- Production Ready
- Easy REST API Development

- Excellent Database Support
- Strong Security Framework
- Huge Ecosystem

### 13. Disadvantages

- More memory than plain Java
- Auto-configuration can hide complexity from beginners
- Large dependency tree
- Startup time is higher than simple Java programs

### 14. Interview Questions

| Question                                       | Answer                                                                                                                                        |
|------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| Q1. What is Spring Boot?                       | A framework built on Spring that simplifies enterprise Java development using auto-configuration, starter dependencies, and embedded servers. |
| Q2. Difference between Spring and Spring Boot? | Spring is the core framework. Spring Boot makes Spring easier by reducing configuration.                                                      |
| Q3. What is Auto Configuration?                | Spring Boot automatically configures components based on the dependencies on the class path.                                                  |
| Q4. Why Embedded Tomcat?                       | No separate server installation. Application contains its own server.                                                                         |
| Q5. What is Maven?                             | A build automation tool that manages dependencies, compiles code, runs tests, and packages the application.                                   |